

Ektron to Optimizely GraphQL Migration

https://gemini.google.com/app/bbb6c13dd19a4c85

User prompt: i am looking for ektron content export samples , so that i can import and organize optimizely then i can generate GraphQL , so when user comes and queries optimizely it has to give response from graphql

Response: To migrate content from **Ektron** to **Optimizely** and serve it via **Optimizely Content Graph (GraphQL)**, you will follow a 3-stage flow: **Export (Ektron XML)** → **Transform & Import (Optimizely CMS)** → **Query (Optimizely Graph)**.

1. Sample Ektron Content Export Format

Ektron exports content primarily as **XML/XLIFF packages** or **SmartForm XML documents**. Below is a representative sample of how Ektron structured content (including metadata and SmartForm fields) appears when exported:

```
<?xml version="1.0" encoding="utf-8"?>
<EktronExportContainer>
  <ContentData>
    <ContentID>1024</ContentID>
    <FolderID>45</FolderID>
    <Title>About Our Enterprise Solutions</Title>
    <Teaser>Leading enterprise software overview</Teaser>
    <LanguageID>1033</LanguageID>
    <DateCreated>2023-04-12T10:30:00Z</DateCreated>
    <Html><![CDATA[<p>Welcome to our digital platform showcase.</p>]]></Html>
    <SmartFormXml>
      <root>
        <Headline>Transforming Legacy Content Systems</Headline>
        <Author>Jane Doe</Author>
        <PublishDate>2023-04-12</PublishDate>
        <HeroImage>/uploadedImages/hero.jpg</HeroImage>
      </root>
    </SmartFormXml>
    <MetaData>
      <Meta name="Keywords">CMS, Migration, Optimizely</Meta>
      <Meta name="Category">Technology</Meta>
    </MetaData>
  </ContentData>
</EktronExportContainer>
```

2. Transforming & Importing into Optimizely CMS

Before Optimizely Graph can index your content, it needs to be organized into **Content Types** (PageTypes, BlockTypes, or ComponentTypes) inside Optimizely CMS.

Step A: Define Content Types in Optimizely

Map Ektron SmartForm fields to strongly typed Optimizely properties:

```
{
  "key": "ArticlePage",
  "baseType": "_page",
  "displayName": "Article Page",
  "properties": [
    { "key": "headline", "type": "String", "displayName": "Headline" },
    { "key": "author", "type": "String", "displayName": "Author" },
    { "key": "mainBody", "type": "XhtmlString", "displayName": "Main Body" },
    { "key": "heroImage", "type": "ContentReference", "displayName": "Hero Image" }
  ]
}
```

Step B: Transform Ektron XML to Optimizely Import Payload

Convert the Ektron export into an Optimizely Content Management REST API payload (or programmatically import using IContentRepository in C# for PaaS installations):

```
{
  "name": "About Our Enterprise Solutions",
  "contentType": "ArticlePage",
  "language": "en",
  "parentLink": { "id": 45 },
  "properties": {
    "headline": "Transforming Legacy Content Systems",
    "author": "Jane Doe",
    "mainBody": "<p>Welcome to our digital platform showcase.</p>",
    "heroImage": "/globalassets/images/hero.jpg"
  }
}
```

3. Automatic GraphQL Schema Generation & Querying

Once content is published or imported into Optimizely CMS, the **Optimizely Content Graph** service automatically indexes the content types and generates corresponding GraphQL schema types. Custom types like `ArticlePage` inherit from `_Page` and `_Content`.

GraphQL Query Sample

When a frontend user or client queries Optimizely Graph, submit requests to the GraphQL endpoint (`https://cg.optimizely.com/content/v2`) (`https://cg.optimizely.com/content/v2`):

```
query GetArticleContent {
  ArticlePage(
    where: {
      _metadata: {
        displayName: { eq: "About Our Enterprise Solutions" }
      }
    }
  ) {
    items {
      _metadata {
        id
        displayName
        published
        url
      }
      headline
      author
      mainBody
      heroImage {
        url
      }
    }
  }
}
```

GraphQL Response Received by User

```
{
  "data": {
    "ArticlePage": {
      "items": [
        {
          "_metadata": {
            "id": 1024,
            "displayName": "About Our Enterprise Solutions",
            "published": "2023-04-12T10:30:00Z",
            "url": "/en/about-our-enterprise-solutions"
          },
          "headline": "Transforming Legacy Content Systems",
          "author": "Jane Doe",
          "mainBody": "<p>Welcome to our digital platform showcase.</p>",

```

```

    "heroImage": {
      "url": "/globalassets/images/hero.jpg"
    }
  }
]
}
}
}

```

Sources

1. [Content base types - Optimizely](#)

User prompt: my requirement is i need to have header menus and footer menus and each menu item will have title and url then i need each map the menu item with the content it holds and if the content mentioned in the menu item not available it has to fallback to default menu item. and one more condition that i query only one id ,lob which should map everything in one graphql query and it should return the response

Response: To fulfill this requirement in Optimizely CMS and Optimizely Content Graph, you need to model your navigation using recursive content references so that a single GraphQL query fetches the **Line of Business (LOB) menu config**, **header items**, **footer items**, **linked content**, and **fallback menu items** in one trip.

1. Optimizely CMS Content Model Architecture

To support LOB-based menus with fallbacks, set up three content types in Optimizely CMS:

1. MenuItemBlock:

- title (*String*) – Custom display title (optional override).
- url (*String*) – External URL or path override.
- targetContent (*ContentReference*) – Link to the page/content.
- fallbackMenuItem (*ContentReference to MenuItemBlock*) – Pointer to the default menu item if targetContent is missing/unpublished.

2. NavigationConfigPage:

- lob (*String*) – Line of Business identifier (e.g., "Banking", "Insurance", "Retail").
- headerMenu (*ContentArea / List of MenuItemBlock*)
- footerMenu (*ContentArea / List of MenuItemBlock*)

2. The Unified Single GraphQL Query

Pass both \$id (the configuration ID) and \$lob (Line of Business) into a single request. The query fetches:

- Header & Footer items.
- Content payload attached to each menu item (targetContent).
- Fallback menu item data (fallbackMenuItem) if the primary content is empty or unlinked.

```

query GetCompleteNavigation($id: String!, $lob: String!) {
  NavigationConfigPage(
    where: {
      _metadata: { id: { eq: $id } }
      lob: { eq: $lob }
    }
  ) {
    items {
      _metadata {
        id
        displayName

```



```

{
  "data": {
    "NavigationConfigPage": {
      "items": [
        {
          "_metadata": {
            "id": "501",
            "displayName": "Retail LOB Navigation"
          },
          "lob": "RETAIL",
          "headerMenu": [
            {
              "title": "Special Offers",
              "url": "/offers",
              "targetContent": {
                "_metadata": {
                  "id": "1002",
                  "url": "/en/offers/summer-sale",
                  "types": ["ArticlePage"]
                },
                "headline": "Summer Clearance Sale",
                "teaser": "Up to 50% off select products."
              },
              "fallbackMenuItem": null
            },
            {
              "title": "LOB Specific Services",
              "url": null,
              "targetContent": null,
              "fallbackMenuItem": {
                "title": "General Services (Default)",
                "url": "/en/services/default-overview",
                "targetContent": {
                  "_metadata": {
                    "id": "1001",
                    "url": "/en/services/default-overview"
                  },
                  "headline": "Welcome to Our Core Services"
                }
              }
            }
          ]
        }
      ],
      "footerMenu": [
        {
          "title": "Privacy Policy",
          "url": "/privacy",
          "targetContent": {
            "_metadata": {
              "id": "900",
              "url": "/en/privacy"
            },
            "headline": "Global Privacy Policy"
          },
          "fallbackMenuItem": null
        }
      ]
    }
  }
}

```

4. Client-Side Resolution Logic

Your frontend application processes the response in a single render pass:

```
interface MenuItem {
  title?: string;
  url?: string;
  targetContent?: Record<string, any> | null;
  fallbackMenuItem?: MenuItem | null;
}

function resolveMenuItem(item: MenuItem) {
  // If content or direct link exists, use primary item
  if (item.targetContent || item.url) {
    return {
      title: item.title || item.targetContent?.headline,
      url: item.url || item.targetContent?._metadata?.url
    };
  }

  // Fallback to default menu item
  if (item.fallbackMenuItem) {
    return {
      title: item.fallbackMenuItem.title || item.fallbackMenuItem.targetContent?.headline,
      url: item.fallbackMenuItem.url || item.fallbackMenuItem.targetContent?._metadata?.url
    };
  }

  return null;
}
```
